A PRACTICAL FIELD GUIDE · 2026

RAG Architectures.

Fourteen retrieval-augmented generation patterns, from the naive baseline to agentic systems: how each one works, where it shines, what it costs, and when to reach for it.

PART A
Foundations
Naive RAG, Advanced RAG, Modular RAG

PART B
Query Intelligence
Multi-Query & RAG-Fusion, HyDE, Adaptive RAG

PART C
Retrieval & Indexing
Hybrid Search RAG, Hierarchical RAG, Graph RAG, Multimodal RAG

PART D
Self-Correcting & Agentic
Self-RAG, Iterative & Multi-hop RAG, Corrective RAG, Agentic RAG

Diagrams, comparison matrix and decision guide included · Ratings are qualitative guidance, not benchmarks

Contents

Fourteen patterns in four families, plus the tools to compare, pick and evaluate them.

Orientation

01 RAG foundations: the baseline pipeline ... 3
02 The landscape: taxonomy and timeline ... 4

Part A · Foundations

01 Naive RAG ... 5
02 Advanced RAG ... 5
03 Modular RAG ... 6

Part B · Query Intelligence

04 Multi-Query & RAG-Fusion ... 7
05 HyDE ... 7
06 Adaptive RAG ... 8

Part C · Retrieval & Indexing

07 Hybrid Search RAG ... 9
08 Hierarchical RAG ... 9
09 Graph RAG ... 10
10 Multimodal RAG ... 10

Part D · Self-Correcting & Agentic

11 Self-RAG ... 11
12 Iterative & Multi-hop RAG ... 12
13 Corrective RAG (CRAG) ... 12
14 Agentic RAG ... 13

Compare, choose, ship

15 Comparison matrix and other variants ... 14
16 Choosing a pattern and the adoption path ... 15
17 Evaluating RAG systems ... 16
18 Production checklist and pitfalls ... 17
19 Glossary and references ... 18

How to read a pattern card

Element	What it tells you
Diagram	The data flow. Dashed lines are optional or lookup paths; amber cylinders are stores; the dark blocks are inputs and outputs; coloured solid blocks are LLM calls.
How it works	The four steps that distinguish this pattern from a plain retrieve-then-read pipeline.
Strengths / Trade-offs	What you gain and what you pay in latency, cost, or complexity.
Best for	The situations where the pattern earns its extra moving parts.
Colour	Blue = foundations, violet = query intelligence, teal = retrieval and indexing, orange = self-correcting and agentic.

01 · RAG foundations

Retrieval-augmented generation gives a language model access to knowledge it was never trained on, by fetching relevant text at question time and placing it in the prompt.

A language model alone answers from its training data: frozen at a cutoff, blind to your private documents, and prone to confident invention. RAG splits the job in two. A **retriever** finds the passages most likely to contain the answer; a **generator** writes a response grounded in those passages. Updating knowledge then means updating an index, not retraining a model, and every answer can point back to its sources.

The baseline pipeline

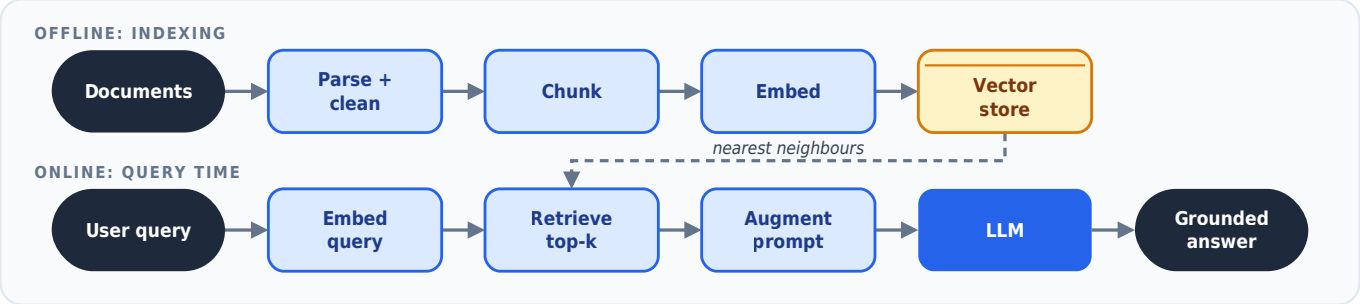


Figure 1. Two phases: an offline indexing pipeline that prepares the knowledge, and an online pipeline that answers each query.

RAG versus the alternatives

Approach	Strong when	Weak when
Prompting only	The model already knows the topic; no private data.	Facts are new, private, or must be cited.
Long context	Small, stable corpus that fits in the window.	Corpora of millions of tokens; per-query cost and latency matter.
Fine-tuning	Teaching style, format, or domain behaviour.	Facts change often; you need traceable sources.
RAG	Large, changing, private knowledge with citations.	Answers need reasoning across the whole corpus (see Graph RAG).

Where basic RAG breaks, and what to reach for

Symptom	Likely cause	Reach for
Misses exact IDs, names, acronyms	Dense-only retrieval	Hybrid Search RAG
Short or vague queries find nothing	Vocabulary mismatch	Multi-Query, HyDE
Right document found, but ranked 20th	Weak first-stage ranking	Reranking (Advanced RAG)
"Summarise the main themes" fails	Chunk-level retrieval only	Hierarchical RAG, Graph RAG
Answer needs facts from 3 documents	Single-shot retrieval	Iterative, Graph, Agentic RAG
Charts and scanned pages are ignored	Text-only index	Multimodal RAG
Confident answers from irrelevant chunks	No retrieval grading	Corrective RAG, Self-RAG
Simple questions are slow and costly	Always retrieving	Adaptive RAG

02 · The landscape

Fourteen patterns fall into four families. Moving right trades simplicity for capability: each step adds latency, cost and failure modes.

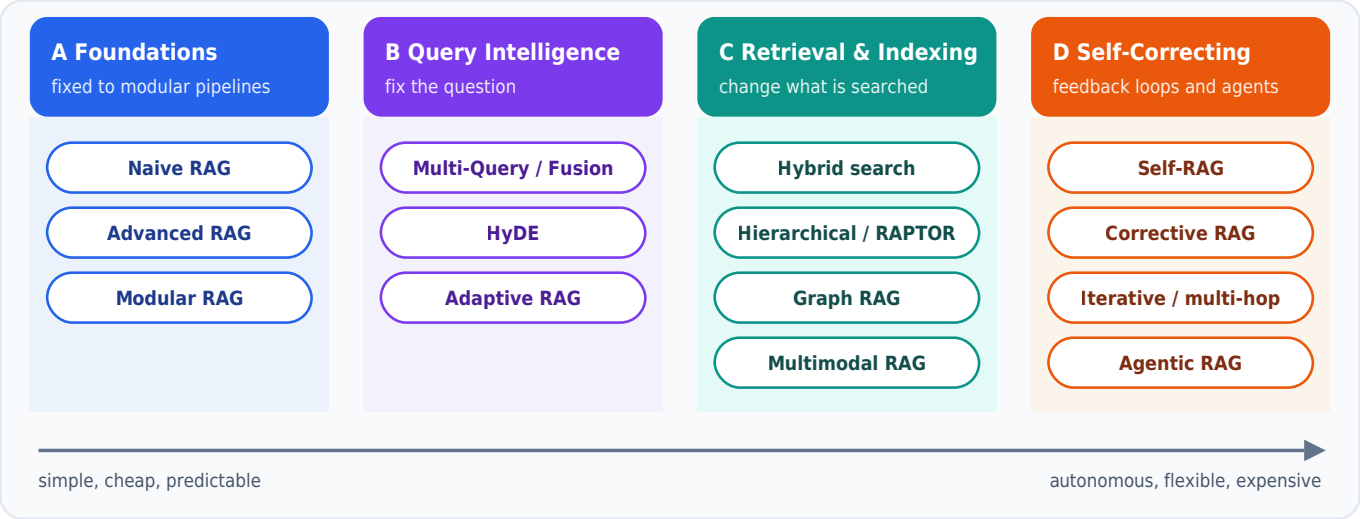


Figure 2. Taxonomy of RAG architectures covered in this guide.

How the field evolved

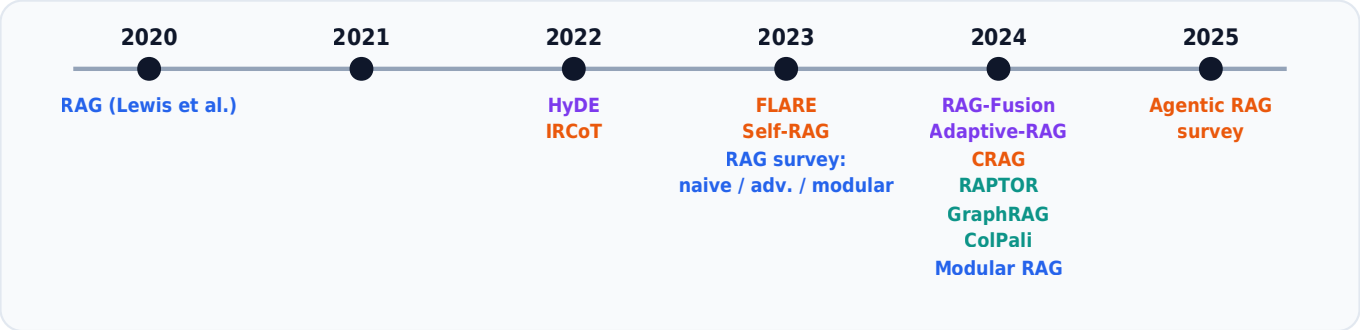


Figure 3. Approximate publication years of the originating papers, coloured by family.

Three ideas that recur in every pattern

Idea	Meaning
Pre-retrieval	Improve what you search with: rewrite, expand, route or imagine the query; or improve what you search *over* with better chunks, metadata, summaries or graphs.
Post-retrieval	Improve what reaches the model: rerank, filter, compress, and grade. Often the cheapest quality gain available.
Control flow	Decide how many times to retrieve and who decides. Pipelines are fixed; routers choose a path; loops and agents choose dynamically.

Rule of thumb. Most production gains come from the first two families and from evaluation, not from autonomy. Reach for Part D when you can name the specific failure it fixes.

A Foundations

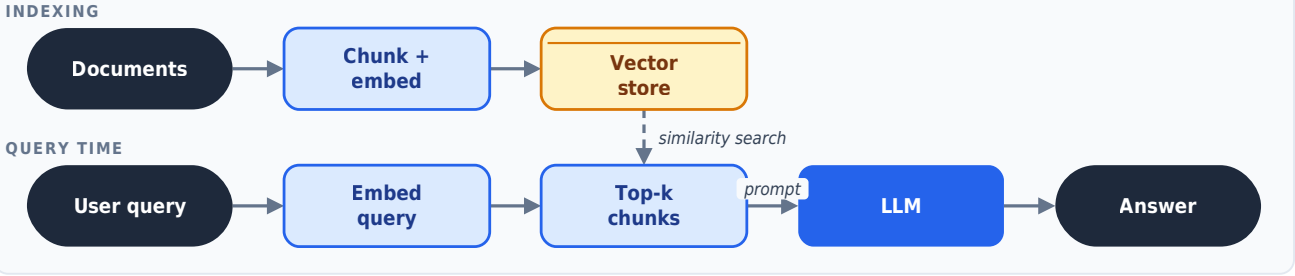
From the fixed retrieve-then-read pipeline to modular designs. Start here, and stay here until your evaluation tells you otherwise.

01 Naive RAG

Index, retrieve, stuff, generate

Lewis et al., 2020

The original recipe: chunk documents, embed them, fetch the top-k nearest chunks for a query, and paste them into the prompt. Every other pattern in this guide exists to fix something this version gets wrong.



HOW IT WORKS

1. Split documents into chunks; embed each into a vector store.
2. Embed the user query with the same model.
3. Fetch the top-k chunks by vector similarity.
4. Insert the chunks into the prompt and generate.

STRENGTHS

+ Fastest to build; one LLM call
+ Cheap and easy to debug

TRADE-OFFS

– Misses exact IDs and synonyms
– Noisy context; no grounding check

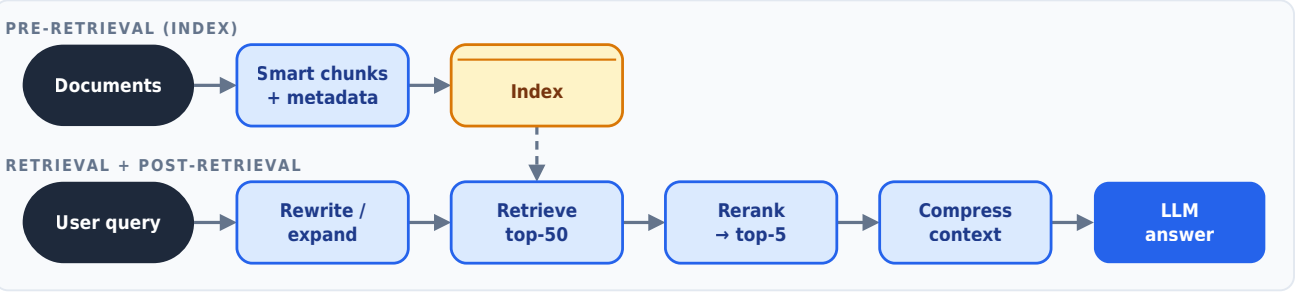
BEST FOR
Prototypes, FAQ bots, small clean corpora.

02 Advanced RAG

Naive RAG + pre- and post-retrieval optimisation

Gao et al., 2023 (survey)

Same retrieve-then-read skeleton, with extra work on both sides of retrieval: smarter chunking and metadata at index time, query rewriting up front, and reranking plus compression before the prompt is built.



HOW IT WORKS

1. Index smarter: structure-aware chunks, overlap, metadata.
2. Rewrite or expand the query before searching.
3. Over-retrieve (say top-50), then rerank with a cross-encoder.
4. Compress and de-duplicate the context, then generate.

STRENGTHS

+ Largest quality gain per unit of effort
+ Reranking sharply lifts precision

TRADE-OFFS

– Added latency from reranking
– More knobs to tune and evaluate

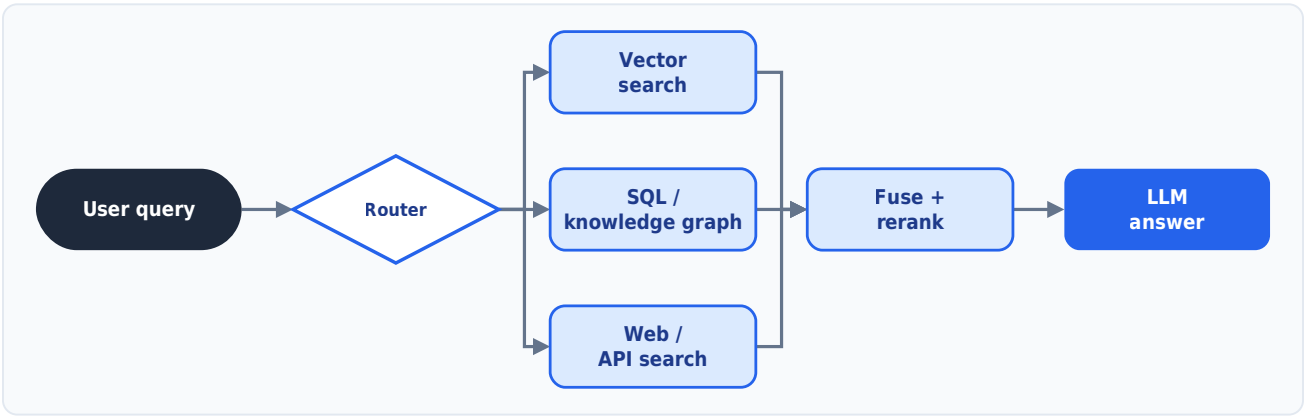
BEST FOR
The default production baseline for knowledge bases and support copilots.

03 Modular RAG

Reconfigurable modules instead of a fixed pipeline

Gao et al., 2024

RAG as a toolkit of swappable modules (routing, search, memory, fusion, rerank, validation) wired together in linear, conditional, branching or looping flows. The pipeline becomes a design decision rather than a given.



HOW IT WORKS

1. Define interchangeable modules behind clear interfaces.
2. Route each query to the sources and flow it needs.
3. Run branches (vector, SQL, web) in parallel and fuse them.
4. Validate, then generate; swap modules without a rewrite.

STRENGTHS

+ Evolve one component at a time
+ Handles heterogeneous sources

TRADE-OFFS

− Orchestration and tracing overhead
− Easy to over-engineer

BEST FOR

Enterprises with several data sources and teams that iterate continuously.

Part A at a glance

Pattern	Core move	Best for
Naive	One shot, vector search only	Prototypes
Advanced	Rewrite + rerank + compress	Production baseline
Modular	Routed, branching, swappable	Many sources and teams

Move from Naive to Advanced first: better chunking and a reranker usually beat any exotic architecture. Adopt Modular only when you genuinely have several sources or flows to orchestrate.

B Query Intelligence

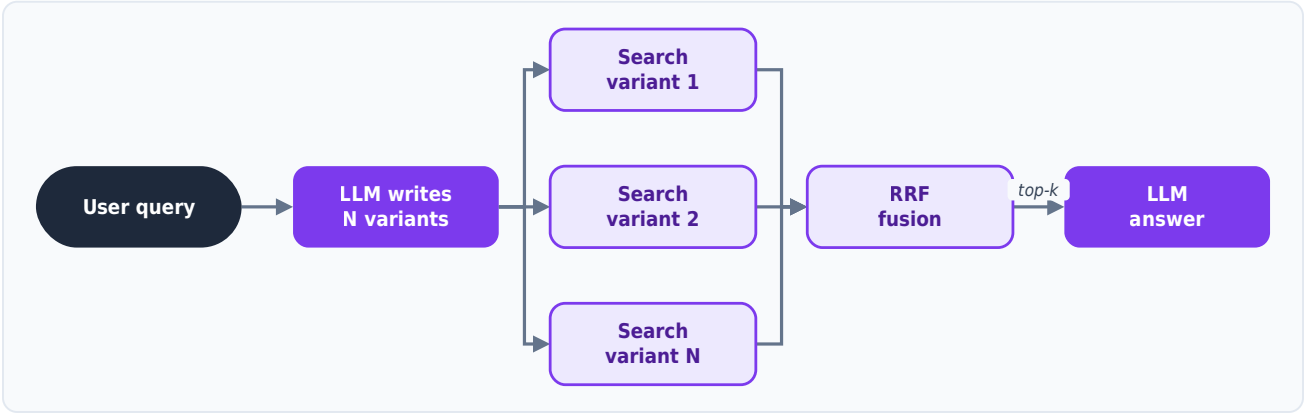
Fix the question before you search. These patterns rewrite, imagine, or route queries so the retriever sees something it can match.

04 Multi-Query & RAG-Fusion

Many phrasings, one merged ranking

Rackauckas, 2024; Cormack et al., 2009

One question rarely matches the best wording of its answer. An LLM writes several variants of the query; each is searched and the ranked lists are merged, typically with Reciprocal Rank Fusion, before generation.



HOW IT WORKS

1. An LLM generates 3-5 reformulations of the query.
2. Retrieve top-k for the original and every variant, in parallel.
3. Merge with RRF: score = Σ 1 / (k + rank), commonly k = 60.
4. Pass the fused top results to the generator.

STRENGTHS

+ Higher recall on vague or short queries
+ Robust to vocabulary mismatch

TRADE-OFFS

− Extra LLM call and N× retrieval
− Variants can drift off-topic

BEST FOR

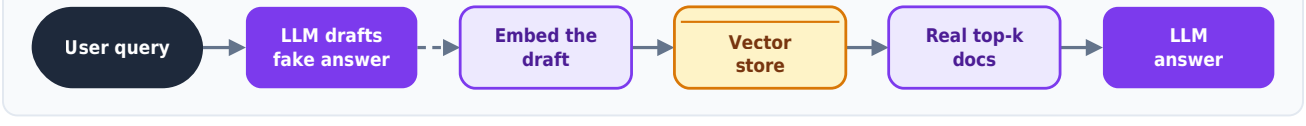
Ambiguous, search-box style questions.

05 HyDE

Search with a fake answer to find real ones

Gao et al., 2022

Hypothetical Document Embeddings. Instead of embedding the question, an LLM writes a plausible (possibly wrong) answer; that passage is embedded and used to find real documents, because answers resemble documents more than questions do.



HOW IT WORKS

1. Prompt an LLM to write a hypothetical answer passage.
2. Embed the passage rather than the query.
3. Retrieve the nearest real documents.
4. Discard the fake text; answer from the retrieved evidence.

STRENGTHS

+ Zero-shot: needs no labelled data
+ Bridges the question-to-document gap

TRADE-OFFS

− Hallucinated details can mislead search
− Weak on topics the LLM doesn't know

BEST FOR

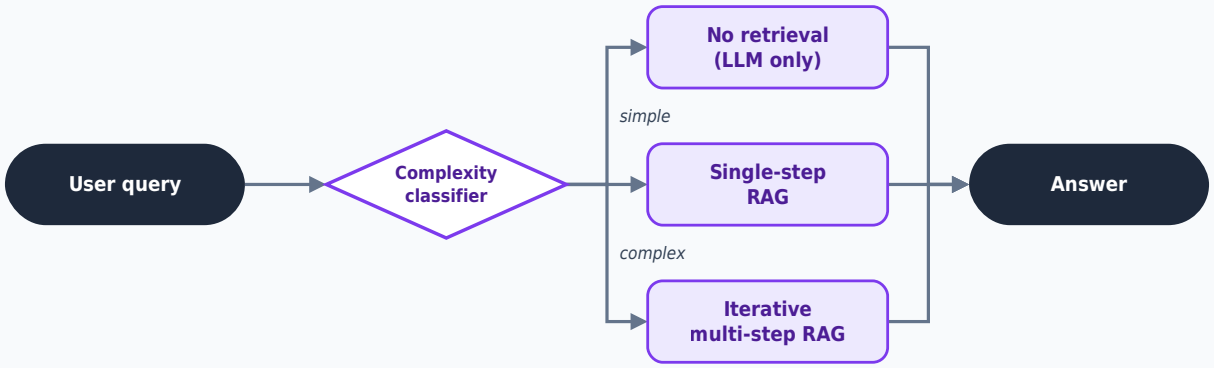
Cold-start domains and short, abstract questions.

06 Adaptive RAG

Match effort to question complexity

Jeong et al., 2024

Not every question deserves retrieval. A lightweight classifier or router inspects the query and picks a strategy: answer directly, run single-step RAG, or launch an iterative multi-step process.



HOW IT WORKS

1. Classify query complexity: simple, moderate, multi-hop.
2. Simple: answer from the LLM alone.
3. Moderate: single-step retrieval and generation.
4. Complex: iterative retrieval interleaved with reasoning.

STRENGTHS

+ Lower average cost and latency
+ Hard questions get the effort they need

TRADE-OFFS

− Router mistakes cascade
− One more component to maintain

BEST FOR

Mixed traffic and cost-sensitive, high-volume apps.

Part B at a glance

Pattern	Core move	Best for
Multi-Query / Fusion	Several phrasings, merged by RRF	Vague, short queries
HyDE	Search with a drafted answer	Cold-start domains
Adaptive	Route by complexity	Mixed traffic, cost control

These patterns are cheap to bolt onto any retriever and combine well: route with Adaptive, then expand the harder queries with Multi-Query or HyDE.

C Retrieval & Indexing

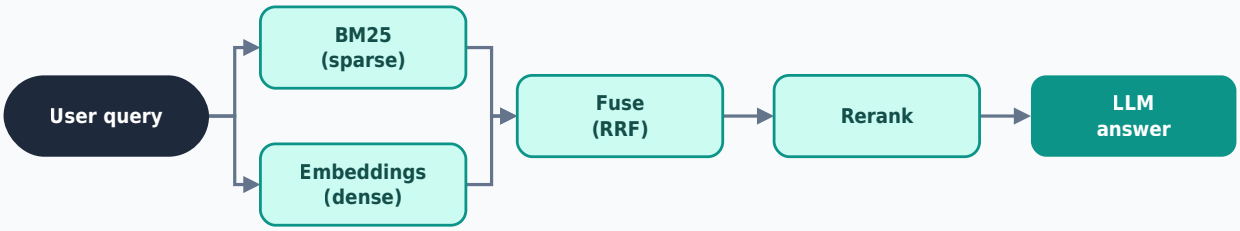
Change what gets indexed and how it is searched: keywords plus vectors, hierarchies of summaries, graphs, and non-text content.

07 Hybrid Search RAG

Keywords and meaning, fused

Standard practice; RRF: Cormack et al., 2009

Dense embeddings capture meaning but stumble on part numbers, names and acronyms; BM25 does the opposite. Hybrid search runs both, fuses the rankings, and usually finishes with a reranker.



HOW IT WORKS

1. Run BM25 (sparse) and vector (dense) search in parallel.
2. Fuse the rankings with RRF or a weighted score.
3. Rerank the merged candidates with a cross-encoder.
4. Generate from the top results.

STRENGTHS

+ Handles exact terms and paraphrases
+ Supported by most vector databases

TRADE-OFFS

– Two indexes to maintain
– Fusion weights need tuning

BEST FOR

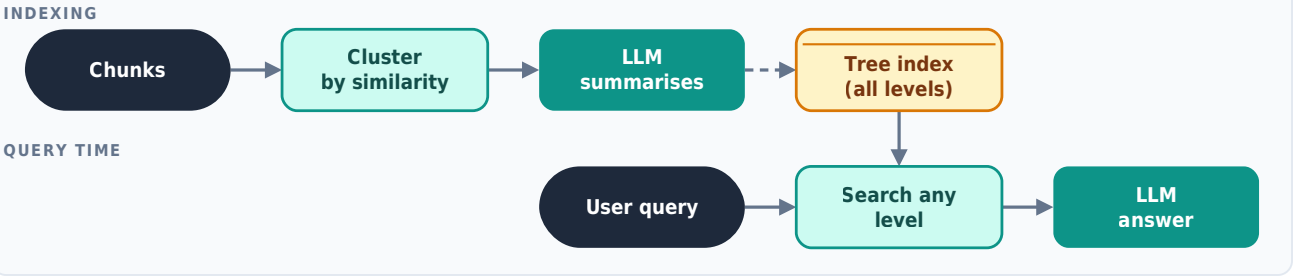
Technical docs, legal, e-commerce, code and log search.

08 Hierarchical RAG

Parent-child retrieval and RAPTOR trees

Sarthi et al., 2024 (RAPTOR)

Two variants, one idea. Parent-child retrieval matches small chunks but returns their larger parent for context. RAPTOR recursively clusters chunks and summarises them into a tree, so a query can land on a detail or on a whole-document theme.



HOW IT WORKS

1. Chunk the text; link children to parents or cluster similar chunks.
2. Summarise clusters upward, layer by layer, into a tree.
3. Index every level and search across all of them.
4. Return matched nodes (or their parents) as context.

STRENGTHS

+ Serves both detail and big-picture questions
+ Fewer fragmented, out-of-context chunks

TRADE-OFFS

– Costly build: many LLM summaries
– Updates force partial re-indexing

BEST FOR

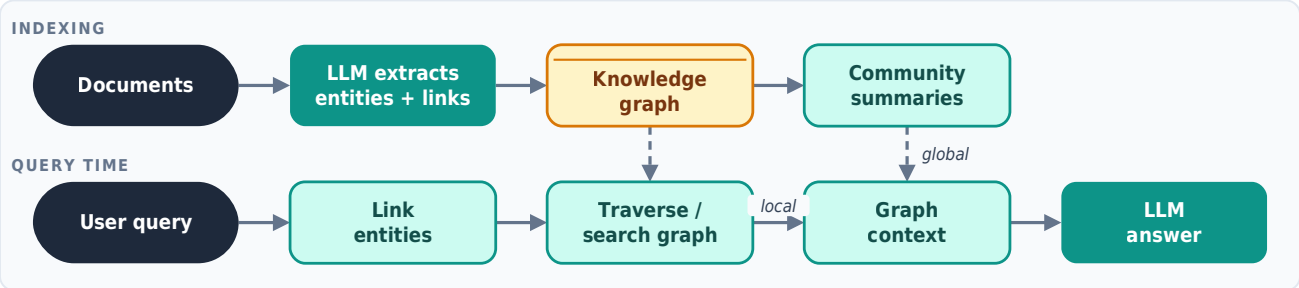
Long reports, books and papers; thematic questions.

09 Graph RAG

Retrieve relationships, not just passages

Edge et al., 2024 (Microsoft GraphRAG)

An LLM extracts entities and relations into a knowledge graph. Queries then traverse neighbourhoods (local search) or use pre-computed community summaries (global search), answering multi-hop and whole-corpus questions that similarity search cannot.



HOW IT WORKS

1. Extract entities and relations from chunks with an LLM.
2. Store them in a graph; detect and summarise communities.
3. Link query entities; traverse the graph (or Text2Cypher).
4. Combine subgraph or summaries as context and generate.

STRENGTHS

+ Strong multi-hop and theme-level answers
+ Explainable evidence paths

TRADE-OFFS

– Expensive extraction and indexing
– Entity resolution quality is critical

BEST FOR

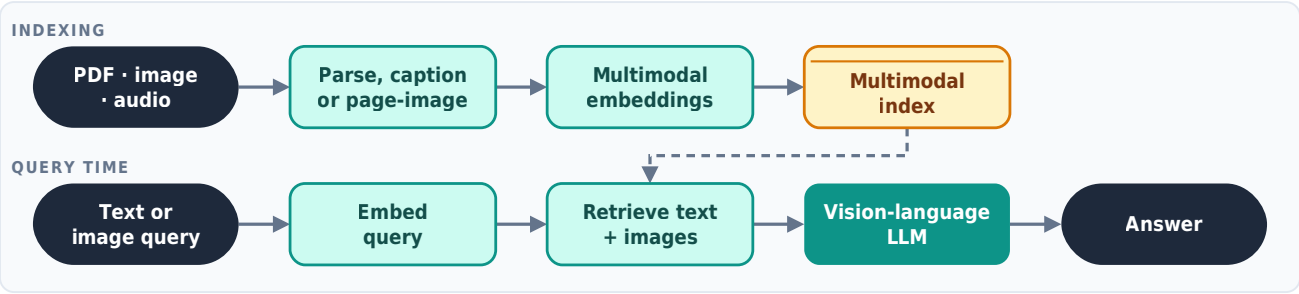
Investigations, biomedical, supply chains, enterprise knowledge graphs.

10 Multimodal RAG

Text, tables, images and audio in one index

Faysse et al., 2024 (ColPali)

Real documents hold charts, tables, diagrams and scans. Multimodal RAG indexes them through shared embeddings, by converting everything to text (captions, table summaries), or by retrieving page images for a vision-language model.



HOW IT WORKS

1. Parse sources into text, tables, images and transcripts.
2. Embed natively (CLIP-style, ColPali) or caption into text.
3. Retrieve across modalities for the query.
4. Generate with a multimodal LLM from text plus images.

STRENGTHS

+ Unlocks charts, slides and scanned PDFs
+ Page-image retrieval skips brittle OCR

TRADE-OFFS

– Larger indexes, higher cost
– Harder to evaluate

BEST FOR

Financial reports, manuals, slide decks.

D Self-Correcting & Agentic

Add feedback loops. The system grades its own retrieval, reasons across several hops, or hands the whole job to a tool-using agent.

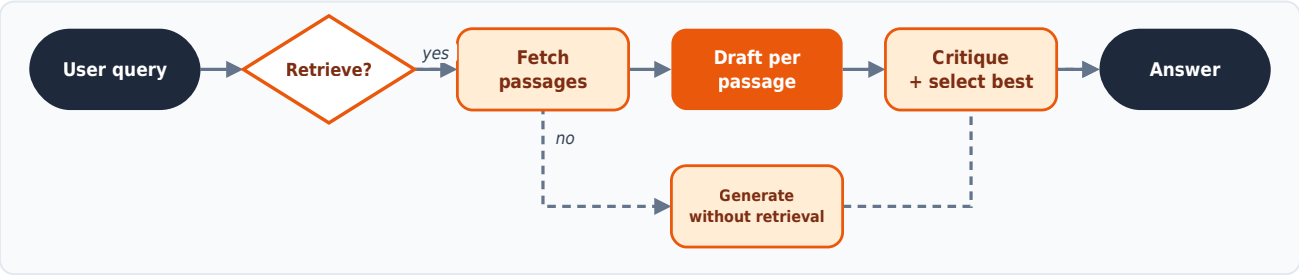
Pattern	Where the loop lives	What gets checked or repeated
Self-RAG	Inside the model	Reflection tokens grade passages and the model's own draft.
Iterative RAG	Around the query	Each result shapes the next sub-query until evidence suffices.
Corrective RAG	Around retrieval	An evaluator grades retrieved documents and triggers repair.
Agentic RAG	Across the system	A planner chooses tools, reflects, and re-plans end to end.

11 Self-RAG

The model decides when to retrieve, then critiques itself

Asai et al., 2023

A language model is trained to emit reflection tokens that decide whether to retrieve and to grade each passage and its own output for relevance, support and usefulness. Generation becomes self-checking.



HOW IT WORKS

1. Decide whether retrieval is needed (Retrieve token).
2. If yes, fetch passages and draft a candidate per passage.
3. Critique each: relevant? supported? useful? (ISREL, ISSUP, ISUSE).
4. Select the best-scoring candidate as the answer.

STRENGTHS

+ Retrieves only when useful
+ Better factuality and citation quality

TRADE-OFFS

– Needs a purpose-trained model
– Several candidates raise cost

BEST FOR

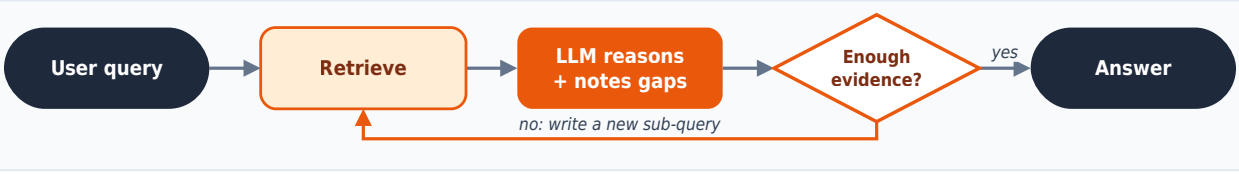
Fact-critical QA and long-form answers with citations.

12 Iterative & Multi-hop RAG

Retrieve, reason, retrieve again

Trivedi et al., 2022 (IRCoT); Jiang et al., 2023 (FLARE)

Questions like "Who founded the company that acquired X?" need evidence from several documents. The system alternates between retrieving and reasoning, using each intermediate result to write the next query until it has enough.



HOW IT WORKS

1. Retrieve with the initial question.
2. Reason over the results and note what is still missing.
3. Form a follow-up query (IRCoT) or re-retrieve when confidence drops (FLARE).
4. Stop when evidence is sufficient, then answer.

STRENGTHS

+ Solves multi-hop questions
+ Adaptive depth; visible reasoning chain

TRADE-OFFS

– Latency grows with each hop
– Errors propagate; needs stop budgets

BEST FOR

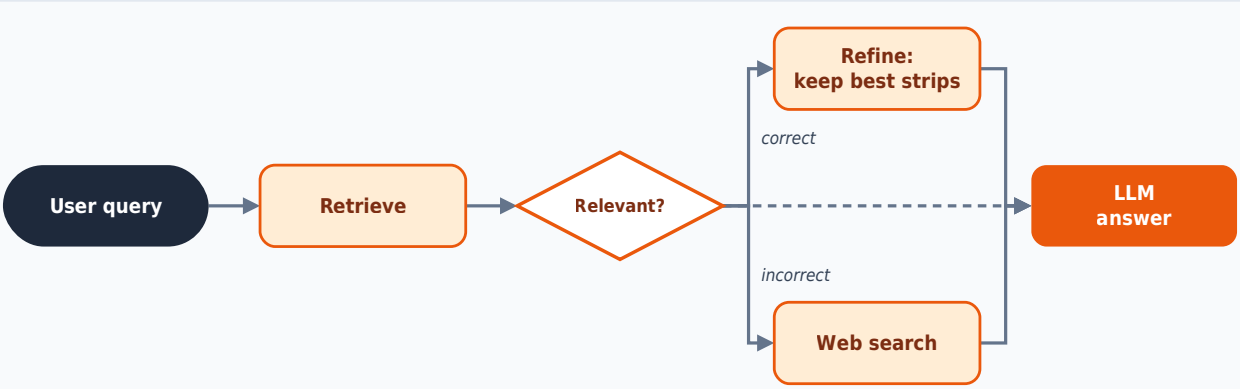
Research questions, comparisons, complex analytics Q&A.

13 Corrective RAG (CRAG)

Grade the retrieval, then repair it

Yan et al., 2024

A lightweight evaluator scores how relevant the retrieved documents are to the query and triggers an action: keep and refine them, discard and search the web, or blend both when the verdict is uncertain.



HOW IT WORKS

1. Retrieve candidate documents from the knowledge base.
2. Evaluator rates confidence: correct, ambiguous or incorrect.
3. Correct: refine. Incorrect: web search. Ambiguous: do both.
4. Generate from the repaired context.

STRENGTHS

+ Guards against bad retrieval
+ Plug-and-play around any retriever

TRADE-OFFS

– Evaluator errors; added latency
– Web results add noise and risk

BEST FOR

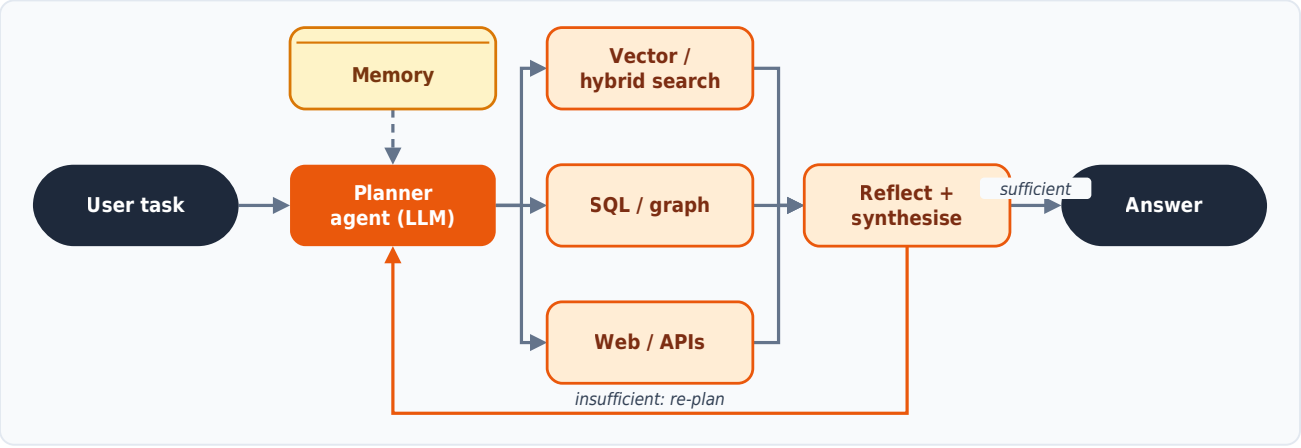
Open-domain assistants and corpora with coverage gaps.

14 Agentic RAG

An agent that plans, uses tools and verifies

Singh et al., 2025 (survey)

Retrieval becomes one tool among many. An LLM agent plans, picks sources (vector search, SQL, graph, web, APIs), calls them, reflects on whether the evidence suffices, and loops or hands off to specialist agents. The most flexible and the most expensive pattern.



HOW IT WORKS

1. A planner agent decomposes the task.
2. It selects and calls tools, reading memory for prior context.
3. Reflect: is the evidence enough and consistent?
4. Re-plan, or synthesise the final answer with citations.

STRENGTHS

+ Handles open-ended, multi-source tasks
+ Self-correcting through reflection

TRADE-OFFS

– Highest cost and latency
– Non-deterministic; needs guardrails

BEST FOR

Research assistants and complex enterprise workflows.

Part D at a glance

Pattern	Core move	Best for
Self-RAG	Model critiques with reflection tokens	Fact-critical answers
Iterative	Retrieve, reason, repeat	Multi-hop questions
Corrective RAG	Grade retrieval, fall back to web	Coverage gaps
Agentic	Planner + tools + reflection	Open-ended tasks

Each loop multiplies latency and cost, so cap hops and budgets. Corrective RAG is the lightest entry point; Agentic RAG is the heaviest and needs tracing and guardrails from day one.

Comparison matrix and other variants

A side-by-side view of the fourteen patterns. More filled dots means more of the attribute.

Pattern	Complexity	Latency	Total cost	Peak quality on hard queries
A Naive RAG	●	●	●	●
A Advanced RAG	●●	●●	●●	●●●
A Modular RAG	●●●●	●●●	●●●	●●●
B Multi-Query & RAG-Fusion	●●	●●●	●●●	●●●
B HyDE	●●	●●●	●●	●●
B Adaptive RAG	●●●	●●	●●	●●●
C Hybrid Search RAG	●●	●●	●●	●●●
C Hierarchical RAG	●●●	●●	●●●●	●●●
C Graph RAG	●●●●●	●●●	●●●●●	●●●●
C Multimodal RAG	●●●●	●●●	●●●●	●●●●
D Self-RAG	●●●●	●●●	●●●	●●●●
D Iterative & Multi-hop RAG	●●●	●●●●	●●●●	●●●●
D Corrective RAG	●●●	●●●	●●●	●●●●
D Agentic RAG	●●●●●	●●●●●	●●●●●	●●●●●

Ratings are judgement-based and relative to each other, not benchmark results. Cost includes index building (summaries, graph extraction) as well as per-query LLM calls. Real numbers depend heavily on your corpus, models and implementation; measure on your own data.

Other variants worth knowing

Variant	Idea
Conversational RAG	Rewrites follow-up questions into standalone queries using chat history before retrieval.
Cache-Augmented Generation	Preloads a small, stable corpus into the model's context or KV cache so no retrieval happens at query time (Chan et al., 2024).
Speculative RAG	A small specialist model drafts answers from document subsets in parallel; a larger generalist verifies and picks one (Wang et al., 2024).
Permission-aware RAG	Filters retrieval by the caller's access rights so users never see passages they cannot open.
Domain-tuned retrievers	Fine-tune the embedding model or reranker on domain question-passage pairs to lift recall without changing the pipeline.

Choosing a pattern

Start from the symptom you can measure, not the pattern you find most interesting.

Your situation	Start with	Then consider
Prototype on a small, clean document set	**Naive RAG**	Advanced RAG once you have an eval set
Part numbers, names, jargon, code	**Hybrid Search RAG**	Reranker
Users type short, vague queries	**Multi-Query / RAG-Fusion**	HyDE for cold-start domains
Mixed easy and hard questions at scale	**Adaptive RAG**	Cache frequent answers
Long reports; "what are the themes?"	**Hierarchical (RAPTOR)**	Graph RAG for cross-document links
Entities and relationships matter	**Graph RAG**	Text2Cypher for structured lookups
Charts, tables, scans, slides	**Multimodal RAG**	Page-image retrieval (ColPali)
Retrieval is often wrong or out of coverage	**Corrective RAG**	Self-RAG if you can train the model
Answers need several linked facts	**Iterative / multi-hop RAG**	Graph RAG
Many tools, sources and open-ended tasks	**Agentic RAG**	Budgets, tracing, human review

A sensible adoption path

Climb one step at a time and keep a step only if your evaluation set shows it earning its cost. Many systems never need to go past step 3.

1. **Naive RAG + an evaluation set**
2. **Better chunking, hybrid search, reranking**
3. **Query rewriting or multi-query where recall is weak**
4. **Routing and retrieval grading (Adaptive, CRAG)**
5. **Hierarchical or graph indexes for global and relational questions**
6. **Iterative or agentic loops, only for tasks that justify them**

Cost warning. Every added LLM call multiplies per-query latency and spend. Graph extraction and hierarchical summaries also add a large one-off indexing bill and a re-indexing cost on every content update.

Evaluating RAG systems

Evaluate retrieval and generation separately, otherwise you cannot tell which half is failing.

Retrieval quality

Metric	Question it answers
Recall@k	Did the right passage appear in the top k?
Precision@k	How much of the top k is relevant?
MRR	How high is the first relevant hit?
nDCG	Is the whole ranking ordered well?
Context recall	Does the context contain everything needed?

Generation quality

Metric	Question it answers
Faithfulness	Is every claim supported by the context?
Answer relevance	Does it address the question asked?
Context precision	Is the context free of noise?
Correctness	Does it match a reference answer?
Abstention	Does it say "I don't know" when it should?

Building an evaluation set

#	Practice
1	Collect **real user questions**, then label the source passages and a reference answer. Start with 50 to 200.
2	**Stratify** by type: simple lookup, multi-hop, thematic, table or figure, and unanswerable. Report each separately; an average hides regressions.
3	Change **one component at a time** (chunking, embedder, reranker, prompt) and re-run the full set.
4	Use an **LLM judge** for faithfulness and relevance, but spot-check it against human labels.
5	Track **latency and cost per query** beside quality, so every pattern is judged on its trade-off.
6	Keep **production traces** and promote failures into the evaluation set.

Tooling

Open-source evaluation libraries include **RAGAS**, **TruLens**, **DeepEval** and **ARES**. For tracing and debugging live traffic, observability tools such as **LangSmith**, **Langfuse** and **Arize Phoenix** record each retrieval and generation step.

Production checklist and pitfalls

What separates a demo that impresses from a system that survives real users.

Checklist before launch

- ☐ Chunk size and overlap tuned on your own documents, not defaults
- ☐ Metadata captured: source, date, section, owner, access level
- ☐ Hybrid search and a reranker in front of the generator
- ☐ Citations returned with every answer, linked to the source passage
- ☐ A clear "not found" path when retrieval confidence is low
- ☐ Access control enforced at retrieval, not only in the UI
- ☐ Index refresh job for new, edited and deleted documents
- ☐ Embedding model versioned; re-index plan for model changes
- ☐ Latency and cost budgets per query, with timeouts and hop limits
- ☐ Caching for frequent queries and repeated embeddings
- ☐ Tracing on every step, with a feedback loop into the eval set
- ☐ Retrieved text treated as untrusted input

Common pitfalls

Pitfall	What to do instead
Prompt injection via documents	Retrieved text can contain instructions. Delimit it, strip active content, and never let it trigger tools unchecked.
Stale or ghost content	Deleted or superseded documents linger in the index and get cited. Sync deletes, not just inserts.
Evaluating on demo questions	A handful of friendly queries hides failures on messy real traffic.
Chunking by character count	Splits tables, lists and definitions mid-way. Respect headings and document structure.
Ignoring the empty result	Without an abstention path the model fills gaps with plausible fiction.
Over-building early	Graphs and agents before a measured baseline make failures impossible to diagnose.

Glossary and references

Key terms used in this guide, and the papers behind each pattern.

Chunk	A slice of a document stored and retrieved as one unit.
Embedding	A vector representing meaning, so similar text lies close together.
Dense / sparse retrieval	Vector similarity versus keyword scoring such as BM25.
ANN / HNSW	Approximate nearest-neighbour search; HNSW is a popular graph-based index.
Cross-encoder reranker	A model scoring query and passage together; slower but more accurate than vectors.
RRF	Reciprocal Rank Fusion: merges ranked lists by summing 1 / (k + rank).
Top-k	The k best-scoring results returned by a search.
Grounding / faithfulness	Whether an answer is supported by the retrieved context.
Knowledge graph	Entities (nodes) and relations (edges) stored as a queryable graph.
Text2Cypher	Generating graph-database queries from natural language.
Multi-hop question	One needing evidence chained across several documents.
Reflection tokens	Special outputs a model uses to critique retrieval and its own text (Self-RAG).

References

1. Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
2. Gao et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey.
3. Gao et al. (2024). Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks.
4. Gao, Ma, Lin, Callan (2022). Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE).
5. Rackauckas (2024). RAG-Fusion: a New Take on Retrieval-Augmented Generation.
6. Cormack, Clarke, Buettcher (2009). Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. SIGIR.
7. Jeong et al. (2024). Adaptive-RAG: Learning to Adapt Retrieval-Augmented LLMs through Question Complexity.
8. Sarthi et al. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR.
9. Edge et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization.
10. Faysse et al. (2024). ColPali: Efficient Document Retrieval with Vision Language Models.
11. Asai et al. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.
12. Yan et al. (2024). Corrective Retrieval Augmented Generation.
13. Trivedi et al. (2022). Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions (IRCoT).
14. Jiang et al. (2023). Active Retrieval Augmented Generation (FLARE).
15. Singh et al. (2025). Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG.
16. Chan et al. (2024). Don't Do RAG: When Cache-Augmented Generation is All You Need for Knowledge Tasks.
17. Wang et al. (2024). Speculative RAG: Enhancing Retrieval Augmented Generation through Drafting.
18. Es et al. (2023). RAGAs: Automated Evaluation of Retrieval Augmented Generation.